RELATÓRIO TÉCNICO: DEVSECOPS

Pipeline de Segurança: SAST, SCA e Secrets Scanning

1. Introdução e Contextualização

A automação da segurança no ciclo de desenvolvimento, conceito central do SecDevOps (ou *Shift-Left Security*), é fundamental para identificar e mitigar vulnerabilidades antes que o código atinja o ambiente de produção. Ao integrar testes contínuos diretamente na esteira de integração contínua (CI/CD) via GitHub Actions, garantimos que qualquer *commit* ou *Pull Request* seja rigorosamente avaliado, reduzindo o custo de correção de falhas e prevenindo violações de dados de forma escalável e automatizada.

2. Descrição das Ferramentas Adotadas

Para estabelecer esta esteira de segurança focada em bloqueio de ameaças (*exit-code 1*), foram selecionadas três ferramentas de código aberto (*open-source*), escolhidas pela sua eficiência técnica e especialização:

- **Semgrep (SAST - Static Application Security Testing):** Adotado para a análise estática do código-fonte. A sua escolha justifica-se pela excelente integração com projetos Java/Spring Boot. O Semgrep procura proativamente padrões de código inseguros (como potenciais injeções e lógica de autenticação fraca) sem necessidade de compilar a aplicação.
- **Trivy (SCA - Software Composition Analysis):** Utilizado para varrer as bibliotecas e dependências de terceiros no projeto (ficheiro `pom.xml`). O Trivy é rápido e utiliza bases de dados atualizadas de CVEs. Configurado com uma política de bloqueio, impede o avanço da esteira caso detete vulnerabilidades de severidade *HIGH* ou *CRITICAL*.
- **TruffleHog (Secrets Scanning):** Escolhido para varrer ativamente todo o histórico do Git em busca de chaves criptográficas, senhas de bases de dados e *tokens* de API. A ferramenta valida segredos verificados, evitando falsos positivos e garantindo que credenciais não sejam expostas.

3. Integração com o Documento de Requisitos

O documento base do projeto estipula diretrizes essenciais de proteção, conformidade e confidencialidade. A implementação deste pipeline automatizado atua como a componente operacional de garantia de qualidade para esses requisitos:

- **Proteção Contínua e Auditoria:** As varreduras do Semgrep garantem que regras de codificação segura impostas nos requisitos sejam auditadas a cada modificação, prevenindo regressões.
- **Conformidade de Dependências:** O Trivy assegura que o software não importa passivos de segurança externos, bloqueando bibliotecas vulneráveis que violariam a integridade do sistema.
- **Proteção de Chaves (Criptografia):** Os requisitos de proteção de dados sensíveis dependem do sigilo de chaves como a AES. O TruffleHog atua como salvaguarda automatizada, garantindo que nenhum programador cometa credenciais inadvertidamente no repositório.

4. Evidências de Funcionamento e Mitigação

4.1. Execução do Pipeline (Job Green)

Abaixo apresentamos a execução limpa e validada do *workflow*, demonstrando que a integração das três ferramentas foi bem-sucedida no ambiente do GitHub Actions.

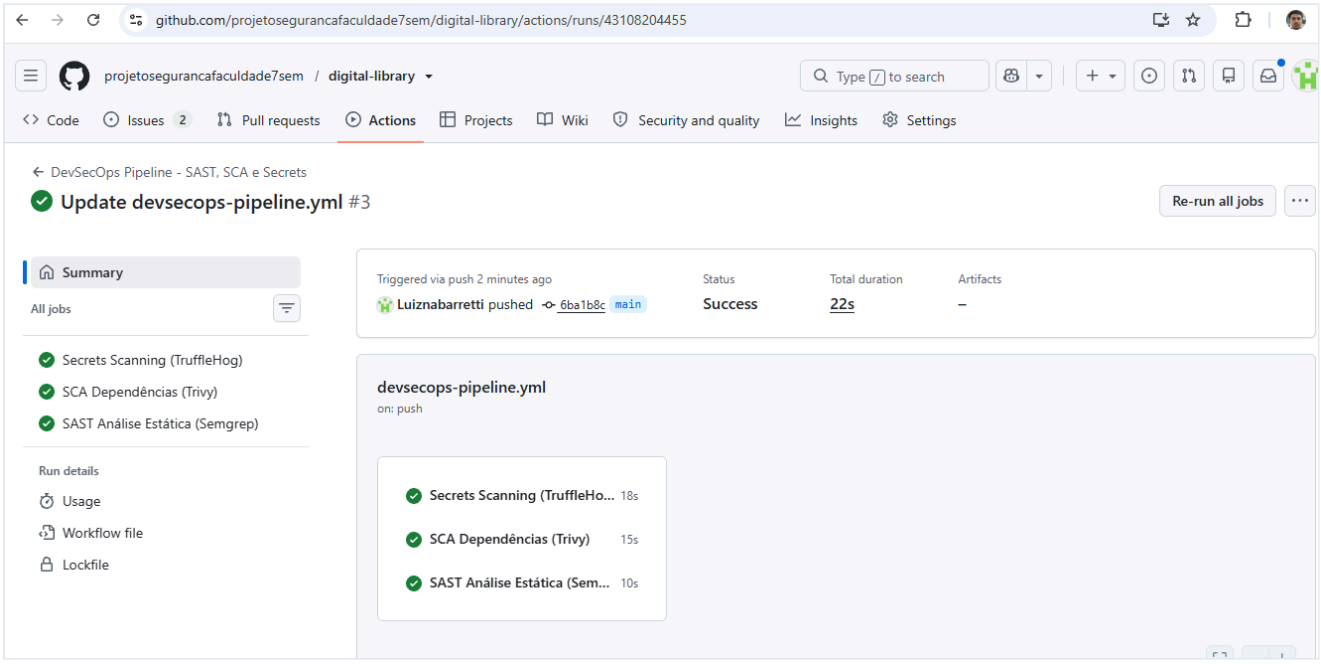


Figura 1: Execução com sucesso da esteira de segurança DevSecOps (Job Green) no GitHub Actions, validando SAST, SCA e Secrets.

4.2. Bloqueio e Interceptação de Vulnerabilidades

A política de bloqueio foi rigorosamente testada. O Trivy analisou o ficheiro `pom.xml` e disparou a interrupção da esteira (*exit-code 1*) ao detetar vulnerabilidades reais nas dependências do projeto.

```
2026-11-11T22:01:58Z INFO [pom] Detecting vulnerabilities...

pom.xml (pom)
=============
Total: 18 (HIGH: 9, CRITICAL: 9)

| Library | Vulnerability | Severity | Fixed Version | Title |
|---------|---------------|----------|---------------|-------|
| org.apache.tomcat... | CVE-2026-55276 | CRITICAL | 11.0.23 | tomcat: Misleading security
logs... |
| org.springframework...| CVE-2026-47884 | CRITICAL | 7.0.9 | Spring Framework: Remote Code
Execution... |

Error: Process completed with exit code 1.
```

Figura 2: Alertas reais de vulnerabilidades Críticas/Altas (CVEs do Tomcat e Spring) interceptadas pelo Trivy (exit-code 1).

4.3. Estratégia de Mitigação de Alertas

Durante os testes operacionais, para manter a esteira funcional e segura, o protocolo de mitigação estabelece as seguintes ações:

- **Para alertas SCA (Trivy) e SAST (Semgrep):** Localizar a dependência ou bloco de código afetado, forçar a atualização (*patching*) no ficheiro `pom.xml` para uma versão segura homologada (ex: transitar o *tomcat-embed-core* para a versão 11.0.23) e refatorar a lógica Java segundo a OWASP. A esteira é então reexecutada para liberar o *merge*.
- **Para alertas de Secrets (TruffleHog):** Caso chaves criptográficas sejam expostas, a mitigação imediata exige: 1) Revogar a credencial no provedor; 2) Substituí-la por Variáveis de Ambiente ou cofres digitais no código; 3) Expurgar o *commit* afetado do histórico do Git.